

Active-Row Subclade Pruning in the `gpurec` Adjoint

gpurec developers

July 13, 2026

Contents

1	Scope and conclusion	1
2	Clade waves and index domains	2
3	How the active mask is constructed	2
4	The reverse recurrence and subclade pruning	3
5	What each masked stage does	3
5.1	Backward-only DTS recomputation: <code>USE_ACTIVE</code>	3
5.2	Within-row adjoint: <code>USE_ACTIVE_MASK</code>	4
5.3	DTS VJP and child-clade propagation	4
5.4	Uninitialized scratch and persistent caches	5
6	The separate split-side mask	5
7	Correctness theorem and proof	5
8	Positive thresholds, error, and Hessian–vector products	6
9	Assumptions and edge cases	6
10	Equation-to-kernel reference	7
11	Practical interpretation	7

1 Scope and conclusion

The flags `USE_ACTIVE` and `USE_ACTIVE_MASK` specialize kernels to skip gene-clade rows whose incoming adjoint is zero or sufficiently small. Despite their short names, they do *not* mask species-tree states. There is one Boolean per gene-clade row in the current wave, and that Boolean applies to all species states of the row.

The mathematical reason the optimization works is row separability. After the forward reconciliation has been fixed, every clade row has its own within-row adjoint system

$$(I - J_i^T)v_i = b_i. \tag{1}$$

If the accumulated right-hand side b_i is exactly zero and the system has a unique solution, then $v_i = 0$. Consequently the row contributes neither parameter cotangents nor cotangents to its child clades. It is therefore exact to omit that row and to stop reverse propagation through that route. Applied in reverse topological order, this can remove a whole descendant subclade.

There is an important qualification. A positive pruning threshold also drops *small nonzero* b_i . That is a deliberate numerical approximation, not an algebraic identity. Exact-zero pruning is exact; threshold pruning is correct as an implementation of the explicitly truncated adjoint and is close to the full gradient only when the discarded cotangents are not strongly amplified by Equation (1). Section 8 states the corresponding bound and assumptions.

2 Clade waves and index domains

Let $\mathcal{C}$ be the gene-clade directed acyclic graph. A recorded split of parent clade i has child clades ℓ and r , so the forward dependency edges are

$$(i, \ell), (i, r) : \quad \Pi_i \text{ depends on } \Pi_\ell \text{ and } \Pi_r. \quad (2)$$

Preprocessing orders clades from leaves toward roots and groups equal-level rows into waves. For a wave beginning at global row $\mathbf{ws}$ and containing W rows, the three relevant domains are

$$w \in \{0, \dots, W-1\}, \quad \text{wave-local clade row,} \quad (3)$$

$$i = \mathbf{ws} + w, \quad \text{global clade row,} \quad (4)$$

$$s \in \{0, \dots, S-1\}, \quad \text{species-tree state.} \quad (5)$$

For split n , `reduce_idx[n]` is its wave-local parent w_n ; the child arrays `sl[n]` and `sr[n]` contain global clade rows. Thus the activity array has shape $[W]$ and is read as

$$a_{w_n} = \text{active_mask}[\text{reduce_idx}[n]]. \quad (6)$$

It is never indexed by s , and it must not be confused with the separate split-side mask of shape $[2N_{\text{split}}]$ discussed in Section 6.

Rows in one wave have no cross-row fixed-point dependence: their children are in earlier waves, while their self-loop is entirely along the species axis of the same row. This makes the within-wave Jacobian block diagonal,

$$J_{\text{wave}} = \text{diag}(J_0, \dots, J_{W-1}), \quad (7)$$

which is the structural fact that permits a single Boolean to remove a whole row from every stage of its local solve.

3 How the active mask is constructed

At the start of the reverse pass, root rows receive the likelihood cotangent. All other entries of the dense array `accumulated_rhs` are initialized to zero. Processing waves from roots toward leaves atomically scatters child cotangents into that array. Immediately before wave k is processed, its slice

$$b_{w,s} = \text{accumulated_rhs}[\mathbf{ws} + \mathbf{w}, \mathbf{s}] \quad (8)$$

therefore contains the *sum of all reverse paths already reaching that clade*. Activity is based on the row infinity norm

$$r_w = \|b_w\|_\infty = \max_s |b_{w,s}|. \quad (9)$$

The kernel `_select_active_adjoint_rows_kernel` scans all S entries and writes one Boolean. The current host helper selects one of two boundary conventions:

$$a_w = \begin{cases} [r_w \geq \tau], & \text{use_pruning} = \text{True}, \\ [r_w > \tau], & \text{use_pruning} = \text{False}, \end{cases} \quad (10)$$

where $[\cdot]$ is an indicator and τ is `adjoint_pruning_threshold`. The distinction is only equality at the threshold. In particular, the current `use_adjoint_pruning=False` path does not make every row active and does not remove the mask; at positive τ it still drops rows with $r_w \leq \tau$. To request the exact-zero criterion with the current interface, use $\tau = 0$ and the strict $>$ branch. To force all rows active through this helper, use $\tau = 0$ and the $\geq$ branch. This detail is an interface fact, not part of the mathematical justification for pruning.

Because Equation (9) uses the accumulated *net* cotangent, exact cancellation between reverse paths may make a row inactive. That is valid: reverse-mode differentiation is linear in cotangents, so a zero net cotangent has a zero downstream VJP even when its individual summands were nonzero.

4 The reverse recurrence and subclade pruning

For each row i , write the converged within-row recurrence abstractly as

$$x_i = F_i(x_i, \{x_j : j \in \text{ch}(i)\}, p), \quad (11)$$

where $x_i \in \mathbb{R}^S$ is the row of Π values, p denotes model parameters, and $\text{ch}(i)$ contains the clades appearing as children in the DTS split terms. At the fixed primal point define

$$J_i = \frac{\partial F_i}{\partial x_i}, \quad C_{ij} = \frac{\partial F_i}{\partial x_j}, \quad B_i = \frac{\partial F_i}{\partial p}. \quad (12)$$

If b_i is the cotangent accumulated from the objective and all already processed parents, implicit reverse differentiation gives

$$v_i = (I - J_i^T)^{-1} b_i, \quad (13)$$

$$b_j += C_{ij}^T v_i \quad (j \in \text{ch}(i)), \quad (14)$$

$$\nabla_p L += B_i^T v_i. \quad (15)$$

The actual kernels decompose $C_{ij}^T v_i$ and $B_i^T v_i$ into duplication, transfer, speciation, extinction, receiver-weight, and leaf terms, but all are linear in the solved cotangent v_i .

If $b_i = 0$, Equation (13) gives $v_i = 0$. Equations (14)–(15) then give zero. Skipping row i is consequently equivalent to evaluating all three equations. Now process the waves in reverse topological order. A child reached only through inactive parents receives no cotangent, is itself classified inactive, and the argument repeats. This is the precise sense in which the optimization prunes a subclade.

The reconciliation structure is a clade DAG, not necessarily a tree. A child may receive cotangents from several parent splits. Inactivating one parent only removes that route. If another active route supplies a sufficiently large cotangent, the child row remains active. Thus “pruned subclade” means the portion of the reverse dependency DAG no longer reachable through active cotangent edges, not blindly every descendant of one inactive split.

5 What each masked stage does

5.1 Backward-only DTS recomputation: `USE_ACTIVE`

The ordinary primal forward pass calls `compute_dts_forward` without an activity array. During the adjoint, DTS values are recomputed for the current wave with `active_parent_rows=active_mask`. This makes `USE_ACTIVE` a compile-time true specialization in three kernels:

- `_reduce_single_gene_split_events_kernel` returns for an inactive parent after writing its output row to $-\infty$ and its gauge offset to zero;

- `_stage_multiple_gene_split_event_reduction_kernel` does not form partial log-sum-exp tiles for an inactive parent;
- `_finalize_multiple_gene_split_event_reduction_kernel` writes the inactive output row to $-\infty$ without reading the uninitialized stage-1 partials.

The output tensor was initialized to $-\infty$ and its offset tensor to zero. More importantly, every later backward load for that row is also guarded by the same activity bit. The sentinel values therefore establish a valid inactive representation; they are not interpreted as a new model value.

5.2 Within-row adjoint: USE_ACTIVE_MASK

The retained wave backward path solves Equation (1) by a Neumann series or GMRES. The mask is applied consistently to every component:

1. `_prepare_reconciliation_self_loop_vjp_kernel` forms event probabilities and the coefficients of J_i^T only for active rows;
2. `_apply_reconciliation_self_loop_transpose_kernel` applies J_i^T only on active rows during every Neumann or GMRES iteration;
3. the transfer receiver-log-weight VJP kernel omits inactive receiver-log-weight cotangents;
4. `_accumulate_reconciliation_event_vjp_kernel` contracts v_i with event probabilities only for active rows and writes zero to its per-row gradient outputs for inactive rows.

For GMRES, the host also explicitly forms

$$b_{w,s}^{\text{GMRES}} = a_w b_{w,s}. \quad (16)$$

The matrix application writes zeros to inactive output rows, so the Krylov problem is exactly the active-row restriction of the block-diagonal system in Equation (7). In the Neumann path, inactive scratch may be deliberately left unwritten to save bandwidth; no mathematical consumer is allowed to read it.

5.3 DTS VJP and child-clade propagation

The kernel `_accumulate_gene_split_event_vjp_kernel` maps each split n to its parent activity using Equation (6). If the parent is inactive, it performs no atomics into child rows and no atomics into shared parameter gradients. Its scalar split-gradient outputs are zeroed when they are requested. In the staged transfer path it writes both split-side activity bits false; when the bandwidth-saving option is enabled, this permits the large per-species side buffers to remain uninitialized safely.

For active parents, the kernel multiplies the solved v_i by normalized DTS event probabilities and scatters those cotangents to the left and right child rows in `accumulated_rhs`. Those atomics are precisely Equation (14). Since the next earlier wave constructs its mask only after all later-wave parents have been processed, its decision sees the complete accumulated child cotangent.

The companion `_accumulate_transfer_subtree_vjp_kernel` checks the same parent-row mask before running the species-tree transfer-complement VJP and before adding receiver-weight or child-row cotangents. The parent activity test and the split-side activity test are cumulative guards: either may skip the work, and neither changes an active, nonzero contribution.

5.4 Uninitialized scratch and persistent caches

Some masked kernels set `SKIP_INACTIVE_SCRATCH_ZERO=True`. Inactive entries of v_i , Neumann work arrays, or staged transfer arrays can therefore contain arbitrary bits, including NaNs. Correctness relies on a strict non-read invariant:

$$a_i = 0 \implies \text{every device read of row } i \text{ is masked, or the row is sanitized first.} \quad (17)$$

The host uses `torch.where`, rather than multiplication by zero, to sanitize v_i before storing a warm start or a Hessian–vector-product cache. This matters because $0 \cdot \text{NaN} = \text{NaN}$. Diagnostic residuals are similarly replaced by zero on inactive rows. The GMRES matrix-output path and per-row parameter-output path explicitly zero inactive rows when their outputs are consumed without a later mask.

6 The separate split-side mask

Transfer-complement VJPs stage two species vectors per gene split, one for each child side. Let $u_{n,q}(s)$ be the staged vector for split n and side $q \in \{L, R\}$. The optional side mask has shape $[2N_{\text{split}}]$ and is

$$z_{n,q} = \begin{cases} [\max_s |u_{n,q}(s)| \neq 0], & \eta = 0, \\ [\sum_s |u_{n,q}(s)| > \eta], & \eta > 0, \end{cases} \quad (18)$$

where η is `pibar_side_threshold`. It can be produced directly by `_accumulate_gene_split_event_vjp_kernel` or by `_select_active_transfer_donor_sides_kernel`. At $\eta = 0$ it skips only an exactly zero linear input, so the species-subtree correction is exactly zero. At a positive η it is another threshold approximation. This mask prunes species-tree work for one transfer side; it does not classify a gene-clade row and is conceptually separate from `USE_ACTIVE_MASK`.

7 Correctness theorem and proof

Proposition (exact-zero active-row pruning). Assume (i) clade waves are in forward topological order; (ii) rows in one wave have no cross-row self-dependence; (iii) every $I - J_i^T$ is non-singular; (iv) the mask declares inactive exactly those rows with $b_i = 0$; and (v) the no-read invariant in Equation (17) holds. Then the masked reverse traversal returns exactly the same child cotangents and parameter gradient as the unmasked traversal, up to the usual floating-point reduction ordering.

Proof. Proceed by induction over waves in reverse topological order. Before the root wave, masked and unmasked traversals have the same likelihood seed. Suppose they have the same accumulated RHS on the current wave. For every active row they run the same local adjoint map and therefore produce the same v_i , child scatters, and parameter contractions. For every inactive row, $b_i = 0$; nonsingularity of $I - J_i^T$ gives the unique solution $v_i = 0$, so the unmasked child and parameter contributions are also zero. Thus both traversals leave identical accumulated RHS values for all earlier waves and identical accumulated parameter gradients. The induction reaches the leaf waves and proves the claim. $\square$

The same argument covers exact-zero split-side pruning because its subsequent tree transformation is linear in $u_{n,q}$. It also explains why shared parameters pose no problem: removing a zero summand leaves their global sum unchanged.

8 Positive thresholds, error, and Hessian–vector products

Let P retain active row blocks and let $e = (I - P)b$ be the discarded RHS. For one row, if $\|J_i\| \leq q_i < 1$ in a compatible operator norm, then

$$\|v_i - v_i^{\text{pruned}}\| = \|(I - J_i^T)^{-1}e_i\| \leq \frac{\|e_i\|}{1 - q_i}. \quad (19)$$

If G_i maps v_i to the row’s parameter and child cotangents, their local error is bounded by

$$\|G_i(v_i - v_i^{\text{pruned}})\| \leq \frac{\|G_i\| \|e_i\|}{1 - q_i}. \quad (20)$$

The infinity-norm test guarantees only $\|e_i\|_\infty < \tau$ (or $\leq \tau$, according to Equation (10)). It does not by itself control $1/(1 - q_i)$, downstream fanout, or the total number of pruned rows. Threshold accuracy therefore depends on contractive, well-conditioned local fixed points and on choosing τ relative to the desired gradient accuracy. The default positive threshold is a performance–accuracy policy, not a proof of bitwise equality with the unpruned gradient.

The exact-HVP path caches the first-order v_i and the same row mask, after sanitizing inactive v_i to zero. Its tangent-adjoint solve and DTS VJP reuse that mask. Consequently it differentiates the algebraic adjoint with the discrete active set held fixed. This is the derivative of the selected piecewise-smooth pruned computation away from a threshold crossing. At $r_i = \tau$ the active set can change discontinuously, and a threshold-pruned HVP should not be interpreted as the Hessian of the smooth, fully unpruned likelihood. With exact-zero pruning, this distinction is normally confined to structurally zero routes; with positive thresholds it is an explicit approximation.

9 Assumptions and edge cases

- A row mask is meaningful only for the wave whose local indexing it uses. Passing a global-clade mask or a species mask violates Equation (6).
- $W = 0$ returns an empty mask. An all-zero root seed makes every downstream row exactly inactive under the strict zero criterion.
- NaN RHS entries are dangerous: comparisons with NaN are false, so a contaminated row can be classified inactive. Finite primal and adjoint states are therefore a precondition, not something masking repairs.
- For $\tau = 0$, the $\geq$ convention marks zero rows active, while the $>$ convention performs exact-zero pruning. For $\tau > 0$ the two conventions differ only at exact equality.
- The local solve is unique when $I - J_i^T$ is nonsingular. Neumann convergence additionally needs a contractive self-loop; GMRES needs the linear system to be solvable. Pruning does not improve a singular active-row system.
- A clade with several incoming reverse routes is tested only after those contributions have been summed. It is not pruned merely because one parent was inactive.
- Floating-point atomics can change last-bit summation order. The exact argument is mathematical; numerical equality is subject to ordinary floating-point effects and the configured finite iterative solve.

10 Equation-to-kernel reference

Kernel or host routine	Activity invariant and mathematical role
<code>compute_active_adjoint_row_mask</code>	Allocates the wave-local $[W]$ Boolean mask and selects the boundary convention in Equation (10).
<code>_select_active_adjoint_rows_kernel</code>	Computes $r_w = \ b_w\ _\infty$ from Equation (9).
<code>compute_dts_forward</code>	Accepts the optional wave-local <code>active_parent_rows</code> ; the ordinary primal leaves it absent, while backward recomputation supplies it.
<code>_reduce_single_gene_split_events_kernel</code>	USE_ACTIVE: skips the one-split DTS value of an inactive parent and publishes the inactive $(-\infty, 0)$ residual/offset pair.
<code>_stage_multiple_gene_split_event_reduction_kernel</code>	USE_ACTIVE: skips all partial split tiles of an inactive multi-split parent.
<code>_finalize_multiple_gene_split_event_reduction_kernel</code>	USE_ACTIVE: avoids reading skipped partials and leaves the inactive DTS row at $-\infty$.
<code>_prepare_reconciliation_self_loop_vjp_kernel</code>	USE_ACTIVE_MASK: forms J_i^T coefficients and the initial local solve state only for active rows.
<code>_apply_reconciliation_self_loop_transpose_kernel</code>	USE_ACTIVE_MASK: applies the block of J_i^T in Equation (1); can explicitly zero inactive outputs when a consumer requires initialized storage.
<code>_accumulate_transfer_receiver_log_probability_vjp_kernel</code>	USE_ACTIVE_MASK: accumulates the transfer-complement receiver VJP only from active v_i .
<code>_accumulate_reconciliation_event_vjp_kernel</code>	USE_ACTIVE_MASK: evaluates Equation (15) and writes zero per-row outputs for inactive rows.
<code>_accumulate_gene_split_event_vjp_kernel</code>	USE_ACTIVE_MASK: reads a_{w_n} using <code>reduce_idx</code> , implements Equation (14), and suppresses every split and parameter scatter from an inactive parent.
<code>_select_active_transfer_donor_sides_kernel</code>	Builds the distinct $[2N_{\text{split}}]$ mask in Equation (18).
<code>_accumulate_transfer_subtree_vjp_kernel</code>	Checks both parent-row and split-side activity before the linear transfer-complement subtree VJP.
<code>implicit_grad_loglik_vjp_wave</code>	Seeds roots, traverses waves in reverse, constructs each mask after RHS accumulation, sanitizes persistent caches, and thereby realizes the induction proof.

11 Practical interpretation

The compile-time flags themselves do not decide which biology is relevant; they merely remove a runtime branch and its guarded arithmetic from a Triton specialization when no mask was supplied. The mathematical decision is the wave-local projection in Equation (10). Exact-zero rows can be removed because the adjoint solve and every outgoing VJP are linear in the incoming cotangent. Positive-threshold rows can be removed because the user has elected to approximate those small cotangents as zero. Reverse wave order, consistent guarding of every consumer, and sanitation of any persistent scratch are what turn that row decision into safe subclade pruning.